EXPERIMENT NO: 01

Aim: Implement a Client/server using RPC/RMI

Theory:

Remote Method Invocation in Java

Remote Method Invocation (RMI) is an API that allows an object to invoke a method on an object that exists in another address space, which could be on the same machine or on a remote machine. Through RMI, an object running in a JVM present on a computer (Client-side) can invoke methods on an object present in another JVM (Server-side). RMI creates a public remote server object that enables client and server-side communications through simple method calls on the server object.

Stub Object: The stub object on the client machine builds an information block and sends this information to the server.

The block consists of

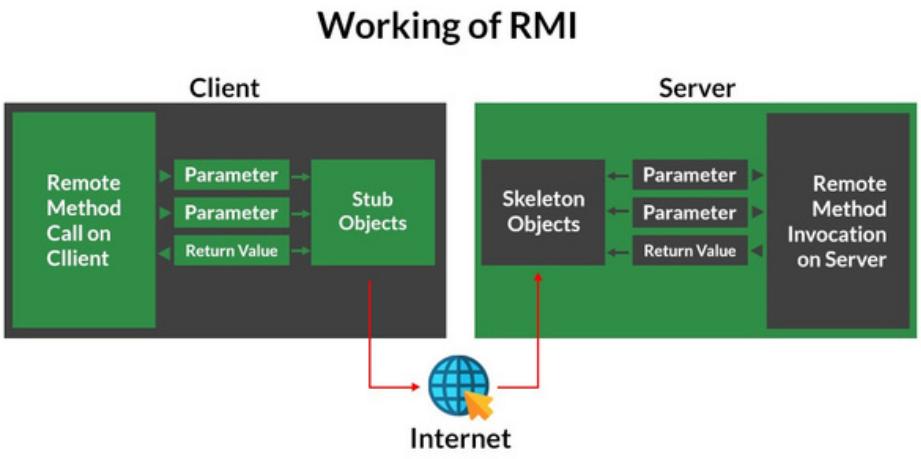
- An identifier of the remote object to be used
- Method name which is to be invoked
- Parameters to the remote JVM

Skeleton Object: The skeleton object passes the request from the stub object to the remote object. It performs the following tasks

- It calls the desired method on the real object present on the server.
- It forwards the parameters received from the stub object to the method.

Working of RMI

The communication between client and server is handled by using two intermediate objects: Stub object (on client side) and Skeleton object (on server-side) as also can be depicted from below media as follows:

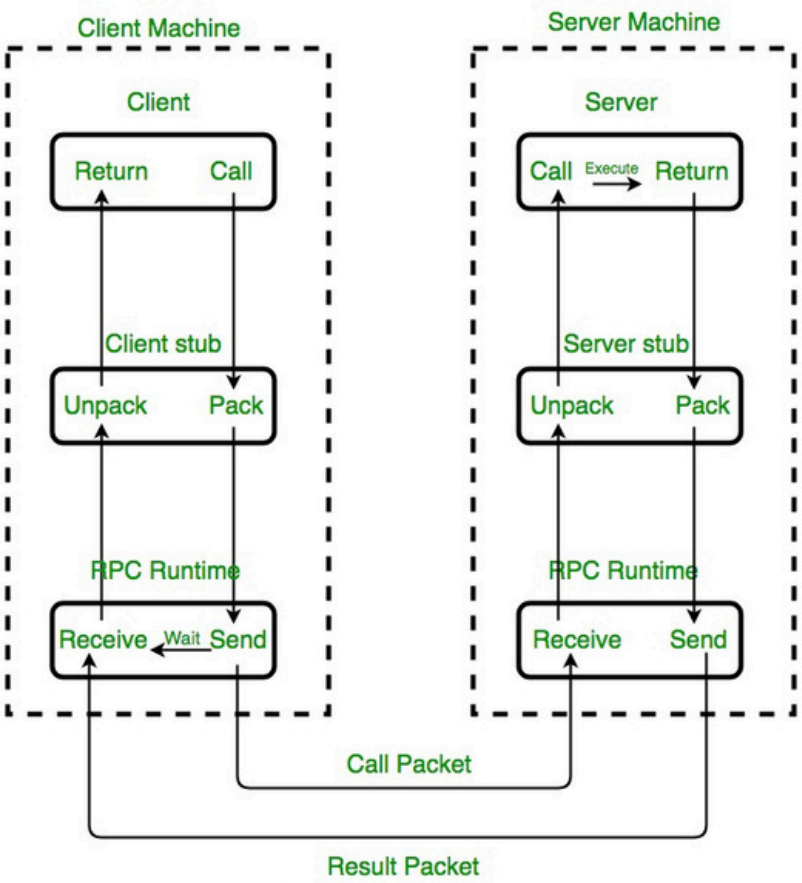


Remote Procedure Call (RPC) is a powerful technique for constructing **distributed, client-server based applications**. It is based on extending the conventional local procedure calling so that the **called procedure does not exist in the same address space as the calling procedure**. The two processes may be on the same system, or they may be on different systems with a network connecting them.

What is Remote Procedure Call (RPC)?

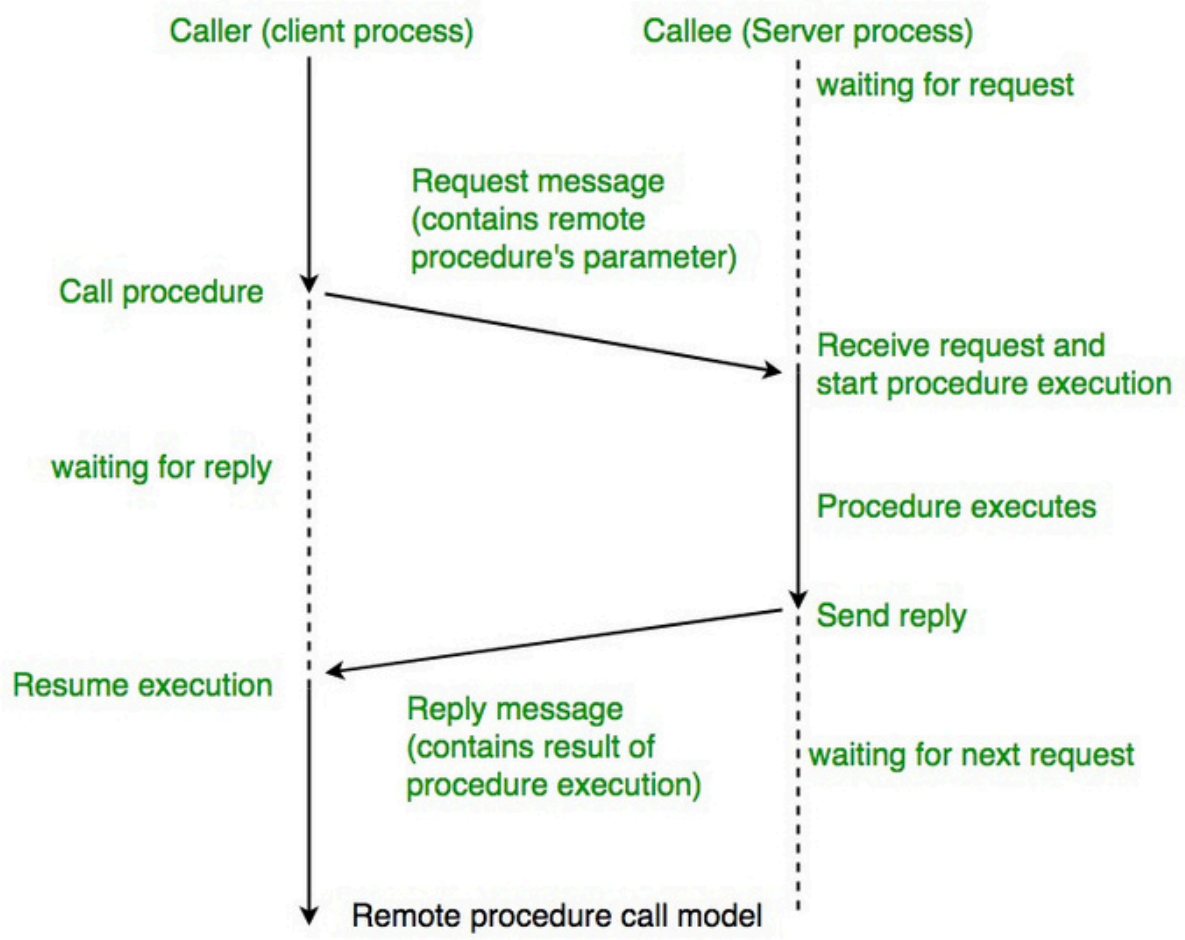
Remote Procedure Call (RPC) is a type of technology used in computing to enable a program to request a service from software located on another computer in a network without needing to understand the network's details. RPC abstracts the complexities of the network by allowing the developer to think in terms of function calls rather than network details, facilitating the process of making a piece of software distributed across different systems.

RPC works by allowing one program (a client) to directly call procedures (functions) on another machine (the server). The client makes a procedure call that appears to be local but is run on a remote machine. When an RPC is made, the calling arguments are packaged and transmitted across the network to the server. The server unpacks the arguments, performs the desired procedure, and sends the results back to the client.



Implementation of RPC mechanism

1. A client invokes a client stub procedure, passing parameters in the usual way. The client stub resides within the client's own address space.
2. The client stub marshalls(pack) the parameters into a message. Marshalling includes converting the representation of the parameters into a standard format, and copying each parameter into the message.
3. The client stub passes the message to the transport layer, which sends it to the remote server machine. On the server, the transport layer passes the message to a server stub, which **demarshalls(unpack)** the parameters and calls the desired server routine using the regular procedure call mechanism.
4. When the server procedure completes, it returns to the server stub **(e.g., via a normal procedure call return)**, which marshalls the return values into a message.
5. The server stub then hands the message to the transport layer. The transport layer sends the result message back to the client transport layer, which hands the message back to the client stub.
6. The client stub demarshalls the return parameters and execution returns to the caller.



client.py server.py

server.py > ...

```python
import socket
def server_program():
    # get the hostname
    host = socket.gethostname()
    port = 5000  # initiate port no above 1024
    server_socket = socket.socket()  # get instance
    # look closely. The bind() function takes tuple as argument
    server_socket.bind((host, port))  # bind host address and port together
    # configure how many client the server can listen simultaneously
    server_socket.listen(2)
    conn, address = server_socket.accept()  # accept new connection
    print("Connection from: " + str(address))
    while True:
        # receive data stream. it won't accept data packet greater than 1024 bytes
        data = conn.recv(1024).decode()
        if not data:
            # if data is not received break
            break
        print("from connected user: " + str(data))
        data = input(' -> ')
        conn.send(data.encode())  # send data to the client
    conn.close()  # close the connection
if __name__ == '__main__':
    server_program()
```

4/36

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

python
pyth...

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS E:\New folder\New folder\DC> python client.py
 -> Rohit Sabat
Received from server: Ragh wardayal ma rya
 ->
```

client.py > ...

```python
import socket
def client_program():
    host = socket.gethostname()  # as both code is running on same pc
    port = 5000  # socket server port number
    client_socket = socket.socket()  # instantiate
    client_socket.connect((host, port))  # connect to the server
    message = input(" -> ")  # take input
    while message.lower().strip() != 'bye':
        client_socket.send(message.encode())  # send message
        data = client_socket.recv(1024).decode()  # receive response
        print('Received from server: ' + data)  # show in terminal
        message = input(" -> ")  # again take input
    client_socket.close()  # close the connection
if __name__ == '__main__':
    client_program()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

python
python

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS E:\New folder\New folder\DC> python client.py
 -> Rohit Sabat
Received from server: Ragh wardayal ma rya
 ->
```

us we have successfully completed a program to implement the

server.py

```
import socket

def server_program():
    # Get the hostname
    host = socket.gethostname()
    port = 5000  # Initiate port no above 1024

    server_socket = socket.socket()  # Get instance

    # The bind() function takes a tuple as an argument
    server_socket.bind((host, port))

    # Configure how many clients the server can listen to simultaneously
    server_socket.listen(2)
    print("Waiting for connection...")

    conn, address = server_socket.accept()  # Accept new connection
    print("Connection from: " + str(address))

    while True:
        # Receive data stream. It won't accept data packets greater than 1024 bytes
        data = conn.recv(1024).decode()
        if not data:
            # If data is not received, break
            break

        print("from connected user: " + str(data))
        data = input(" -> ")
        conn.send(data.encode())  # Send data to the client

    conn.close()  # Close the connection

if __name__ == '__main__':
    server_program()
```

client.py

```
import socket

def client_program():
    host = socket.gethostname()  # As both codes are running on same PC
    port = 5000  # Socket server port number

    client_socket = socket.socket()  # Instantiate
    client_socket.connect((host, port))  # Connect to the server

    message = input(" -> ")  # Take input

    while message.lower().strip() != 'bye':
        client_socket.send(message.encode())  # Send message
        data = client_socket.recv(1024).decode()  # Receive response

        print('Received from server: ' + data)  # Show in terminal
        message = input(" -> ")  # Again take input

    client_socket.close()  # Close the connection

if __name__ == '__main__':
    client_program()
```

Conclusion: Thus we have successfully completed a program to implement the RPC/RMI mechanism.